



assignment4.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help



Share



🔍 Commands + Code ▾ + Text ▶ Run all ▾

RAM
Disk 

Exploratory Data Analysis (EDA), Data Preprocessing, and Visualization



```
import pandas as pd
```

```
# Google Sheet URL (ensure it's shareable and publicly acces
google_sheet_url = "https://docs.google.com/spreadsheets/d/1
```

```
# Convert the Google Sheet URL to a CSV export URL
csv_export_url = google_sheet_url.replace("/edit?usp=sharing
```

```
# Load the dataset into a Pandas DataFrame
try:
```

```
    df = pd.read_csv(csv_export_url)
    print("Dataset loaded successfully!")
    print(df.head())
```

```
except Exception as e:
    print(f"Error loading dataset: {e}")
    print("Please ensure the Google Sheet is published to th
```

```
Dataset loaded successfully!
```

	X1 transaction date	X2 house age	X3 distance to the near
0	2012.917	32.0	
1	2012.917	19.5	
2	2013.583	13.3	
3	2013.500	13.3	
4	2012.833	5.0	

	X4 number of convenience stores	X5 latitude	X6 longitude
0	10.0	24.98298	121.54024
1	9.0	24.98034	121.53951
2	5.0	24.98746	121.54391
3	5.0	24.98746	121.54391
4	5.0	24.97937	121.54245

	Y house price of unit area
0	37.9
1	42.2
2	47.3
3	54.8
4	43.1

```
print("First five rows of the dataset:")
print(df.head(5))
```

```
print("\nLast three rows of the dataset:")
print(df.tail(3))
```



First five rows of the dataset:

	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
0	2012.917	32.0	104.81010	10.0	24.98298	121.54024	37.9
1	2012.917	19.5	90.45606	9.0	24.98034	121.53951	42.2
2	2013.583	13.3	355.00000	5.0	24.98746	121.54391	47.3
3	2013.500	13.3		5.0	24.98746	121.54391	54.8
4	2012.833	5.0		5.0	24.97937	121.54245	43.1

Last three rows of the dataset:

	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
412	2013.000	8.1	104.81010		24.96674	121.54067	52.5
413	2013.500	6.5	90.45606		24.97433	121.54310	63.9
414	2013.167	1.9	355.00000		24.97293	121.54026	40.5

```
print(f"The dataset has {df.shape[0]} rows and {df.shape[1]} columns")
```

The dataset has 415 rows and 7 columns.

```
print("Column names:")
print(df.columns)
```

```
Column names:
Index(['X1 transaction date', 'X2 house age',
       'X3 distance to the nearest MRT station',
       'X4 number of convenience stores', 'X5 latitude', 'X6 longitude',
       'Y house price of unit area'],
      dtype='object')
```

```
print("Descriptive statistics for numerical attributes:")
print(df.describe())
```

```
Descriptive statistics for numerical attributes:
X1 transaction date  X2 house age  \
```

	X1 transaction date	X2 house age
count	415.000000	415.000000
mean	2013.149014	17.674458
std	0.281628	11.405161
min	2012.667000	0.000000
25%	2012.917000	8.950000
50%	2013.167000	16.100000
75%	2013.417000	28.100000
max	2013.583000	43.800000

	X3 distance to the nearest MRT station
count	415.000000
mean	1082.129338
std	1261.092057
min	23.382840
25%	289.324800
50%	492.231300
75%	1452.760000
max	6488.021000

	X4 number of convenience stores	X5 latitude	X6 longitude
count	414.000000	415.000000	415.00
mean	4.094203	24.969039	121.53
std	2.945562	0.012397	0.01
min	0.000000	24.932070	121.47
25%	1.000000	24.963010	121.52
50%	4.000000	24.971100	121.53
75%	6.000000	24.977450	121.54
max	10.000000	25.014590	121.56

	Y house price of unit area
count	415.000000
mean	37.986265
std	13.590608
min	7.600000
25%	27.700000
50%	38.500000
75%	46.600000
max	117.500000

```
print("\nDataset Information (Data Types and Non-Null Counts):
df.info()

print("\nNumber of Missing Values Per Column:")
print(df.isnull().sum())
```

```
Dataset Information (Data Types and Non-Null Counts):
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 415 entries, 0 to 414
Data columns (total 7 columns):
#   Column                                     Non-Null Count
---  ---
0   X1 transaction date                       415 non-null
1   X2 house age                             415 non-null
2   X3 distance to the nearest MRT station   415 non-null
3   X4 number of convenience stores          414 non-null
4   X5 latitude                             415 non-null
5   X6 longitude                             415 non-null
```

```

5     X6 longitude                                415 non-null
6     Y house price of unit area                  415 non-null
dtypes: float64(7)
memory usage: 22.8 KB

Number of Missing Values Per Column:
X1 transaction date      0
X2 house age             0
X3 distance to the nearest MRT station  0
X4 number of convenience stores      1
X5 latitude              0
X6 longitude             0
Y house price of unit area          0
dtype: int64

```

```

df['X22'] = df['X2 house age'] * 365
print("Added 'X22' column (house age in days). Here are the
print(df[['X2 house age', 'X22']].head())

```

```

Added 'X22' column (house age in days). Here are the first 5

```

	X2 house age	X22
0	32.0	11680.0
1	19.5	7117.5
2	13.3	4854.5
3	13.3	4854.5
4	5.0	1825.0

```

df = df.drop('X22', axis=1)
print("Column 'X22' deleted. Here are the first 5 rows of the
print(df.head())

```

```

Column 'X22' deleted. Here are the first 5 rows of the DataFr

```

	X1 transaction date	X2 house age	X3 distance to the near
0	2012.917	32.0	
1	2012.917	19.5	
2	2013.583	13.3	
3	2013.500	13.3	
4	2012.833	5.0	

	X4 number of convenience stores	X5 latitude	X6 longitude
0	10.0	24.98298	121.54024
1	9.0	24.98034	121.53951
2	5.0	24.98746	121.54391
3	5.0	24.98746	121.54391
4	5.0	24.97937	121.54245

	Y house price of unit area
0	37.9
1	42.2
2	47.3
3	54.8
4	43.1

```

import pandas as pd

# Create three new synthetic instances
new instances data = {

```

```

    'X1 transaction date': [2013.25, 2013.75, 2012.75],
    'X2 house age': [15.0, 5.0, 30.0],
    'X3 distance to the nearest MRT station': [250.0, 1500.0
    'X4 number of convenience stores': [8.0, 2.0, 5.0],
    'X5 latitude': [24.98000, 24.97000, 24.99000],
    'X6 longitude': [121.54000, 121.53000, 121.55000],
    'Y house price of unit area': [45.0, 30.0, 50.0]
}

```

```
new_instances_df = pd.DataFrame(new_instances_data)
```

```

# Append the new instances to the original DataFrame
df = pd.concat([df, new_instances_df], ignore_index=True)

```

```

print("Three new synthetic instances have been added to the
print("Displaying the tail of the DataFrame to show the new
print(df.tail())

```

Three new synthetic instances have been added to the dataset.
Displaying the tail of the DataFrame to show the new instance

	X1 transaction date	X2 house age \		X3 distance to the nearest MRT station	X4 number of con
413	2013.500	6.5		90.45606	
414	2013.167	1.9		355.00000	
415	2013.250	15.0		250.00000	
416	2013.750	5.0		1500.00000	
417	2012.750	30.0		600.00000	

	X5 latitude	X6 longitude	Y house price of unit area
413	24.97433	121.54310	63.9
414	24.97293	121.54026	40.5
415	24.98000	121.54000	45.0
416	24.97000	121.53000	30.0
417	24.99000	121.55000	50.0

```

df = df.iloc[:-3]
print("The three newly inserted instances have been deleted.
print("Displaying the tail of the DataFrame to confirm the r
print(df.tail())

```

The three newly inserted instances have been deleted.
Displaying the tail of the DataFrame to confirm the removal:

	X1 transaction date	X2 house age \		X3 distance to the nearest MRT station	X4 number of con
410	2012.667	5.6			
411	2013.250	18.8			
412	2013.000	8.1			
413	2013.500	6.5			
414	2013.167	1.9			

```

410          90.45606
411          390.96960
412          104.81010
413          90.45606
414          355.00000

```

```

      X5 latitude  X6 longitude  Y house price of unit area
410      24.97433      121.54310              50.0
411      24.97923      121.53986              40.6
412      24.96674      121.54067              52.5
413      24.97433      121.54310              63.9
414      24.97293      121.54026              40.5

```

```

print("Original maximum house price of unit area:", df['Y ho

# Update 'Y house price of unit area' to 110 wherever the cu
df.loc[df['Y house price of unit area'] > 110, 'Y house pric

print("Updated maximum house price of unit area:", df['Y hou
print("\nDataFrame head after updating house prices:")
print(df.head())

```

Original maximum house price of unit area: 117.5

Updated maximum house price of unit area: 110.0

DataFrame head after updating house prices:

```

      X1 transaction date  X2 house age  X3 distance to the near
0          2012.917          32.0
1          2012.917          19.5
2          2013.583          13.3
3          2013.500          13.3
4          2012.833           5.0

```

```

      X4 number of convenience stores  X5 latitude  X6 longitude
0              10.0      24.98298      121.54024
1               9.0      24.98034      121.53951
2               5.0      24.98746      121.54391
3               5.0      24.98746      121.54391
4               5.0      24.97937      121.54245

```

```

      Y house price of unit area
0              37.9
1              42.2
2              47.3
3              54.8
4              43.1

```

```

print("Houses with 'Y house price of unit area' <= 20:")
houses_low_price = df[df['Y house price of unit area'] <= 20]
print(houses_low_price[['X5 latitude', 'X6 longitude']])

```

Houses with 'Y house price of unit area' <= 20:

```

      X5 latitude  X6 longitude
8          24.95095      121.48458
40         24.94155      121.50381
41         24.94297      121.50342
48         24.94684      121.49578

```

49	24.94925	121.49542
55	24.94968	121.53009
73	24.94155	121.50381
83	24.96056	121.50831
87	24.94297	121.50342
93	24.94920	121.53076
113	24.96172	121.53812
116	24.94375	121.47883
117	24.93885	121.50383
155	24.94155	121.50381
156	24.94883	121.52954
162	24.94297	121.50342
170	24.94741	121.49628
176	24.94867	121.49507
180	24.94898	121.49621
183	24.94155	121.50381
226	24.94155	121.50381
229	24.94890	121.53095
231	24.94235	121.50357
232	24.95032	121.49587
249	24.95743	121.47516
251	24.94960	121.53018
255	24.95095	121.48458
298	24.94155	121.50381
309	24.94883	121.52954
320	24.93885	121.50383
329	24.93885	121.50383
330	24.94935	121.53046
331	24.94826	121.49587
347	24.95719	121.47353
384	24.94297	121.50342
409	24.94155	121.50381



```
print("Number of missing values in 'X4 number of convenience stores'")
print(df['X4 number of convenience stores'].isnull().sum())

# Calculate the average of 'X4 number of convenience stores'
average_convenience_stores = df['X4 number of convenience stores'].mean()

# Replace missing values with the average
df['X4 number of convenience stores'].fillna(average_convenience_stores, inplace=True)

print("\nNumber of missing values in 'X4 number of convenience stores'")
print(df['X4 number of convenience stores'].isnull().sum())

print("\nDataFrame info after imputation:")
df.info()
```

```
Number of missing values in 'X4 number of convenience stores'
1
```

```
Number of missing values in 'X4 number of convenience stores'
0
```

```
DataFrame info after imputation:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 415 entries, 0 to 414
```

Data columns (total 7 columns):

#	Column	Non-Null Count
0	X1 transaction date	415 non-null
1	X2 house age	415 non-null
2	X3 distance to the nearest MRT station	415 non-null
3	X4 number of convenience stores	415 non-null
4	X5 latitude	415 non-null
5	X6 longitude	415 non-null
6	Y house price of unit area	415 non-null

dtypes: float64(7)

memory usage: 22.8 KB

/tmp/ipykernel_1786/781191629.py:8: FutureWarning: A value is
The behavior will change in pandas 3.0. This inplace method w

For example, when doing 'df[col].method(value, inplace=True)'

```
df['X4 number of convenience stores'].fillna(average_conven
```

```
import numpy as np

# Original column
distance_col = df['X3 distance to the nearest MRT station']

print("Original 'X3 distance to the nearest MRT station' (first 5 rows)")
print(distance_col.head())
print("\n")

# (a) Z-score normalization
mean_distance = distance_col.mean()
std_distance = distance_col.std()
df['X3_distance_zscore'] = (distance_col - mean_distance) / std_distance
print("Z-score Normalized 'X3 distance to the nearest MRT station' (first 5 rows)")
print(df['X3_distance_zscore'].head())
print("Mean: ", df['X3_distance_zscore'].mean())
print("Standard Deviation: ", df['X3_distance_zscore'].std())
print("\n")

# (b) Min-Max normalization
min_distance = distance_col.min()
max_distance = distance_col.max()
df['X3_distance_minmax'] = (distance_col - min_distance) / (max_distance - min_distance)
print("Min-Max Normalized 'X3 distance to the nearest MRT station' (first 5 rows)")
print(df['X3_distance_minmax'].head())
print("Min: ", df['X3_distance_minmax'].min())
print("Max: ", df['X3_distance_minmax'].max())
print("\n")

# (c) Decimal scaling
# Find the maximum absolute value to determine 'j'
max_abs_distance = abs(distance_col).max()
j = len(str(int(max_abs_distance))) if max_abs_distance > 0 else 1

df['X3_distance_decimal'] = distance_col / (10**j)
```

```
print("Decimal Scaled 'X3 distance to the nearest MRT station'")
print(df['X3_distance_decimal'].head())
print("Max Absolute Value: ", df['X3_distance_decimal'].abs()).

# Displaying the new columns along with the original for comparison
print("\nComparison of original and normalized columns (first 5 rows):")
print(df[['X3 distance to the nearest MRT station', 'X3_distance_decimal', 'X3_distance_zscore', 'X3_distance_minmax']])
```

```
Original 'X3 distance to the nearest MRT station' (first 5 rows):
0      84.87882
1     306.59470
2     561.98450
3     561.98450
4     390.56840
Name: X3 distance to the nearest MRT station, dtype: float64
```

```
Z-score Normalized 'X3 distance to the nearest MRT station' (first 5 rows):
0     -0.790783
1     -0.614971
2     -0.412456
3     -0.412456
4     -0.548383
Name: X3_distance_zscore, dtype: float64
Mean:  4.280377926265664e-18
Standard Deviation:  1.0000000000000002
```

```
Min-Max Normalized 'X3 distance to the nearest MRT station' (first 5 rows):
0      0.009513
1      0.043809
2      0.083315
3      0.083315
4      0.056799
Name: X3_distance_minmax, dtype: float64
Min:  0.0
Max:  1.0
```

```
Decimal Scaled 'X3 distance to the nearest MRT station' (first 5 rows):
0      0.008488
1      0.030659
2      0.056198
3      0.056198
4      0.039057
Name: X3_distance_decimal, dtype: float64
Max Absolute Value:  0.6488020999999999
```

```
Comparison of original and normalized columns (first 5 rows):
```

	X3 distance to the nearest MRT station	X3_distance_zscore	X3_distance_minmax	X3_distance_decimal
0	84.87882	-0.790783	0.009513	0.008488
1	306.59470	-0.614971	0.043809	0.030659
2	561.98450	-0.412456	0.083315	0.056198
3	561.98450	-0.412456	0.083315	0.056198
4	390.56840	-0.548383	0.056799	0.039057


```

numerical_cols = ['X1 transaction date', 'X2 house age', 'X3
                  'X4 number of convenience stores', 'Y house
sns.pairplot(df[numerical_cols], diag_kind='kde')
plt.suptitle('Pair Plot of Numerical Attributes', y=1.02) #
plt.show()

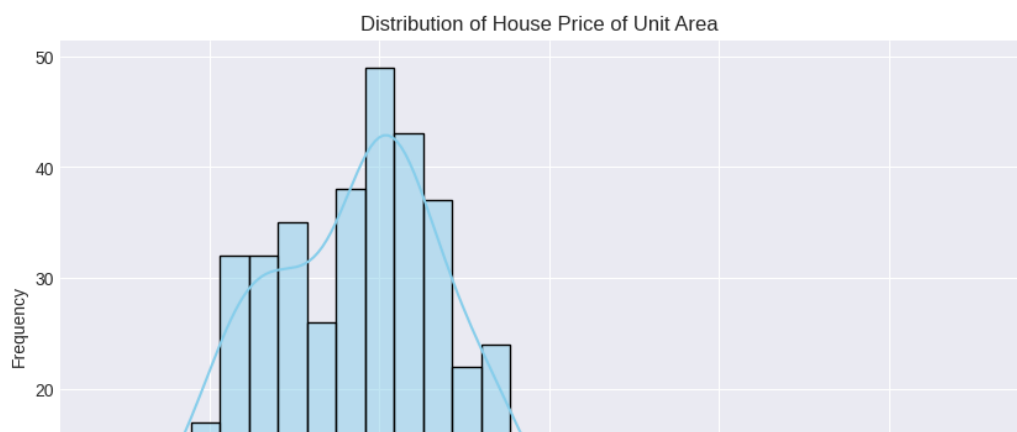
# (g) Plot the distribution (KDE or histogram with KDE) of t
plt.figure(figsize=(10, 6))
sns.histplot(df['Y house price of unit area'], kde=True, col
plt.title('Distribution of House Price of Unit Area (Histo
plt.xlabel('House Price of Unit Area')
plt.ylabel('Density/Frequency')
plt.show()

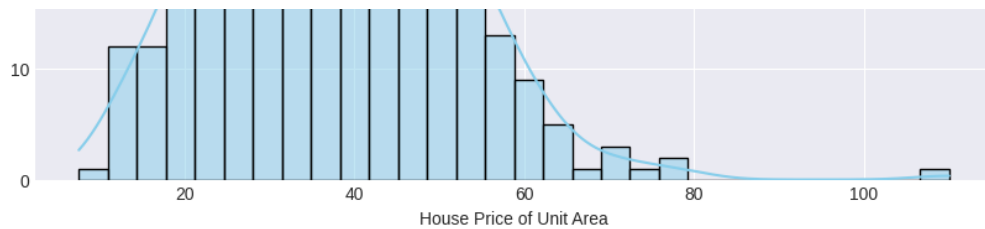
# (h) Create a count plot showing the distribution of the nu
plt.figure(figsize=(10, 6))
sns.countplot(x='X4 number of convenience stores', data=df,
plt.title('Distribution of Number of Convenience Stores')
plt.xlabel('Number of Convenience Stores')
plt.ylabel('Count')
plt.show()

# (i) Create a bar chart showing the average house price for
plt.figure(figsize=(10, 6))
average_price_by_stores = df.groupby('X4 number of convenien
sns.barplot(x='X4 number of convenience stores', y='Y house
plt.title('Average House Price by Number of Convenience Stor
plt.xlabel('Number of Convenience Stores')
plt.ylabel('Average House Price of Unit Area')
plt.show()

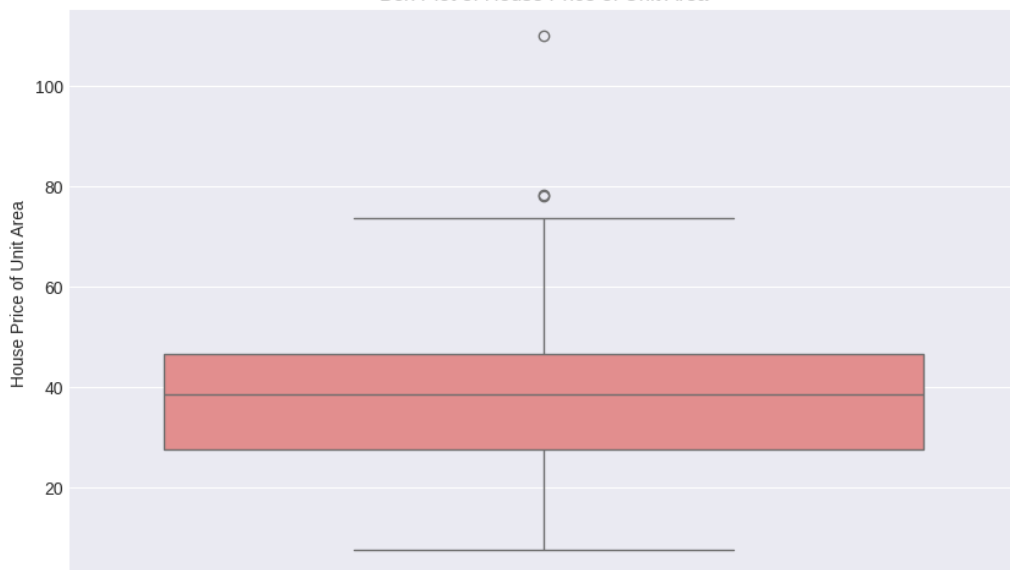
# (j) Draw a line plot showing the relationship between hous
plt.figure(figsize=(12, 6))
# Group by house age and calculate mean price, then sort by
average_price_by_age = df.groupby('X2 house age')['Y house p
average_price_by_age = average_price_by_age.sort_values('X2
sns.lineplot(x='X2 house age', y='Y house price of unit area
plt.title('Relationship Between House Age and Average House
plt.xlabel('House Age (years)')
plt.ylabel('Average House Price of Unit Area')
plt.grid(True)
plt.show()

```

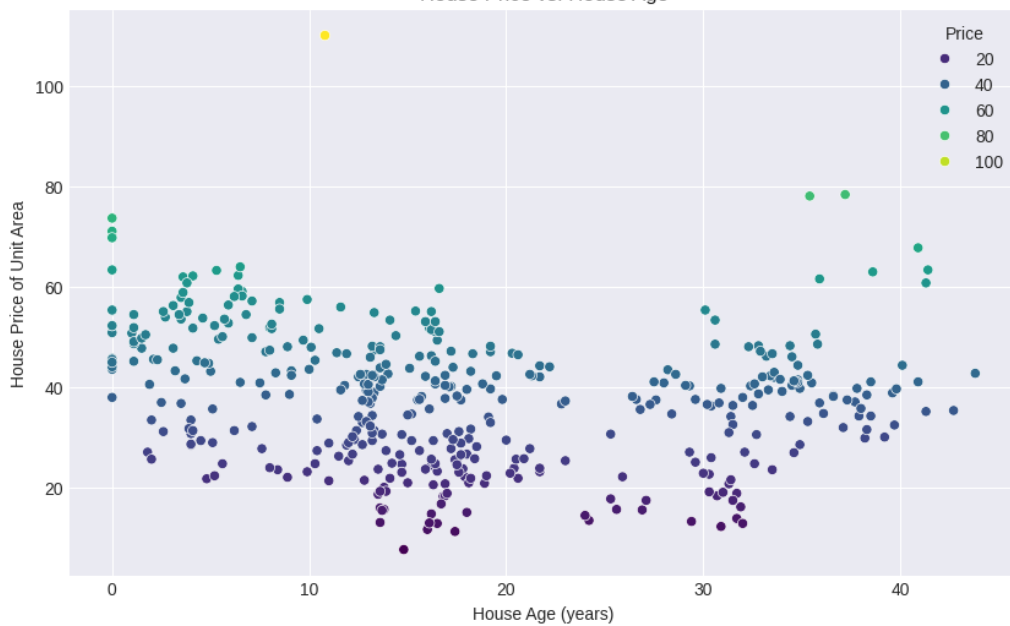




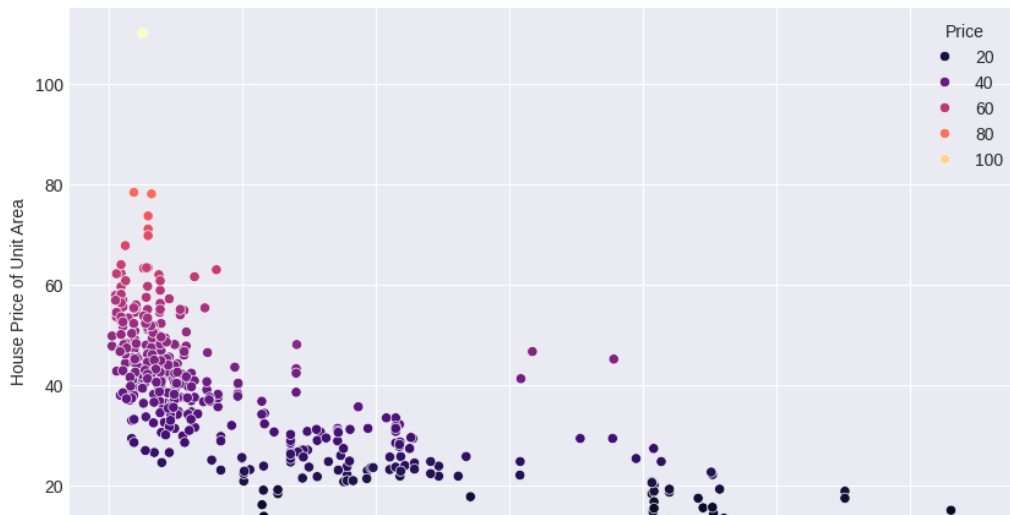
Box Plot of House Price of Unit Area

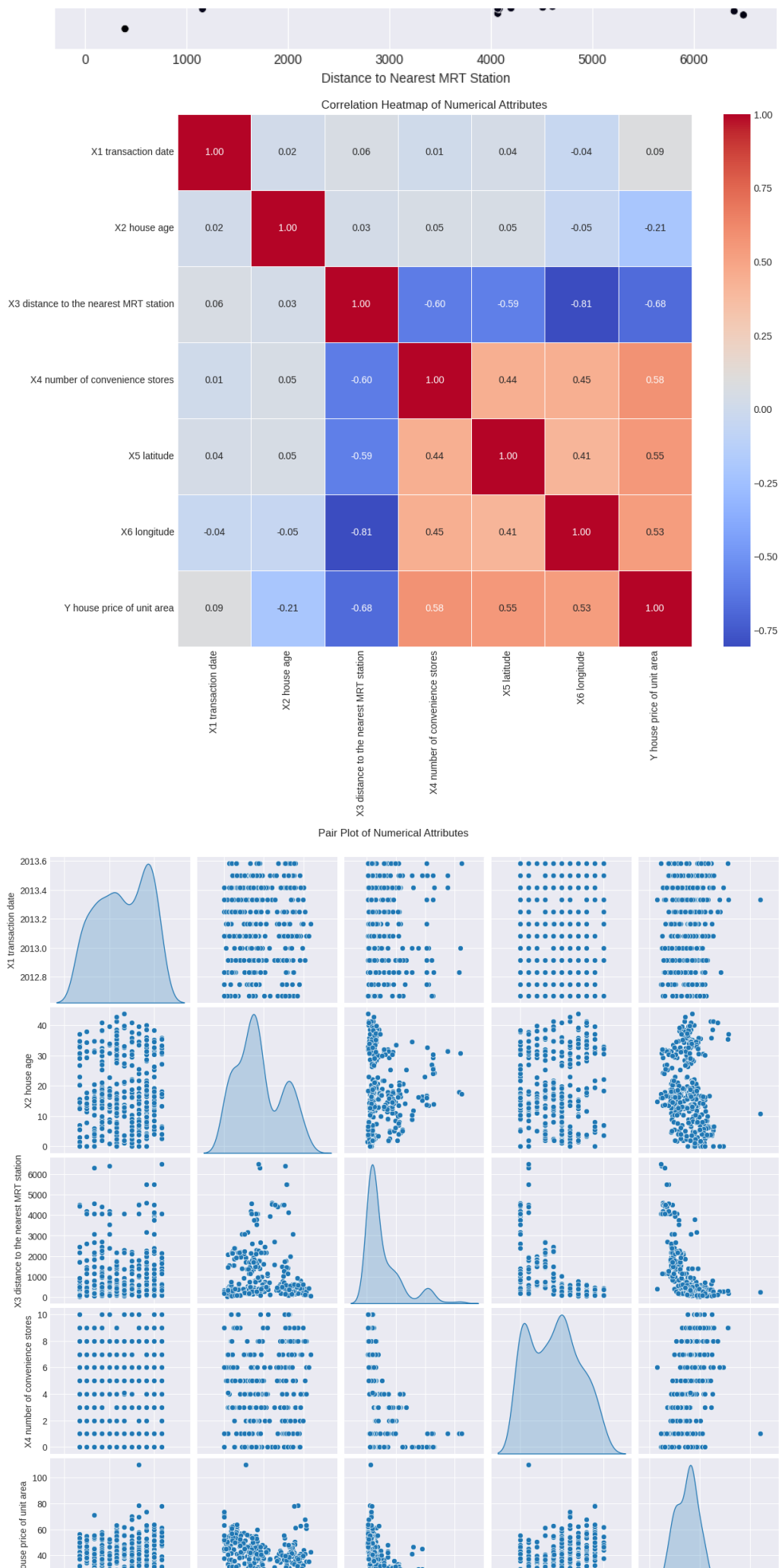


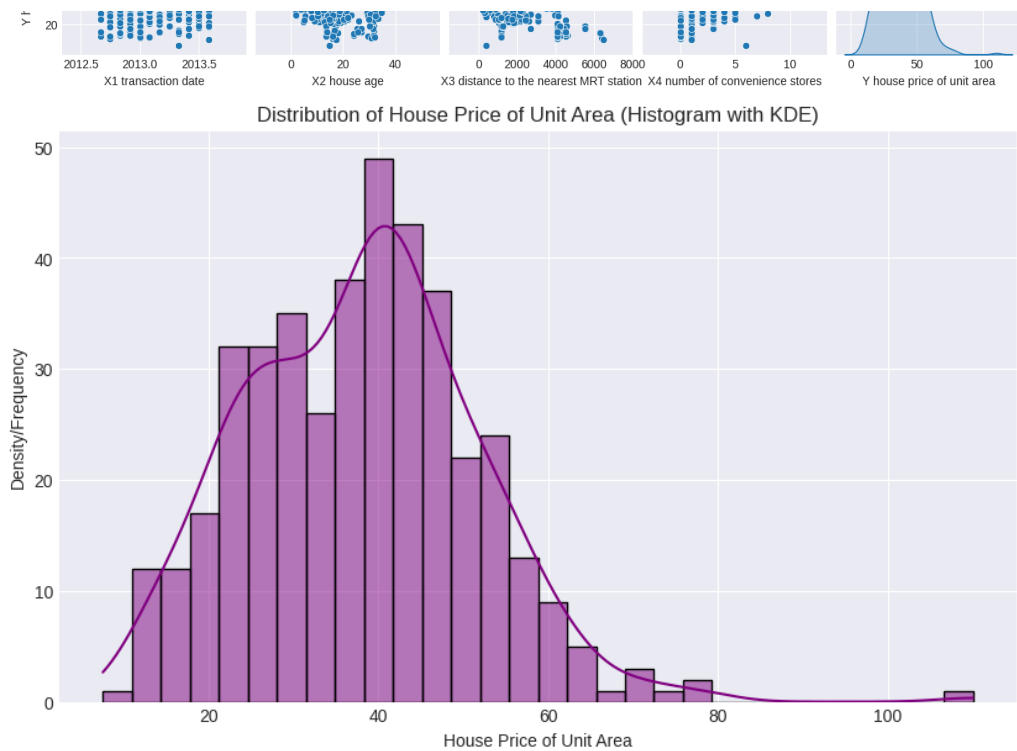
House Price vs. House Age



House Price vs. Distance to Nearest MRT Station



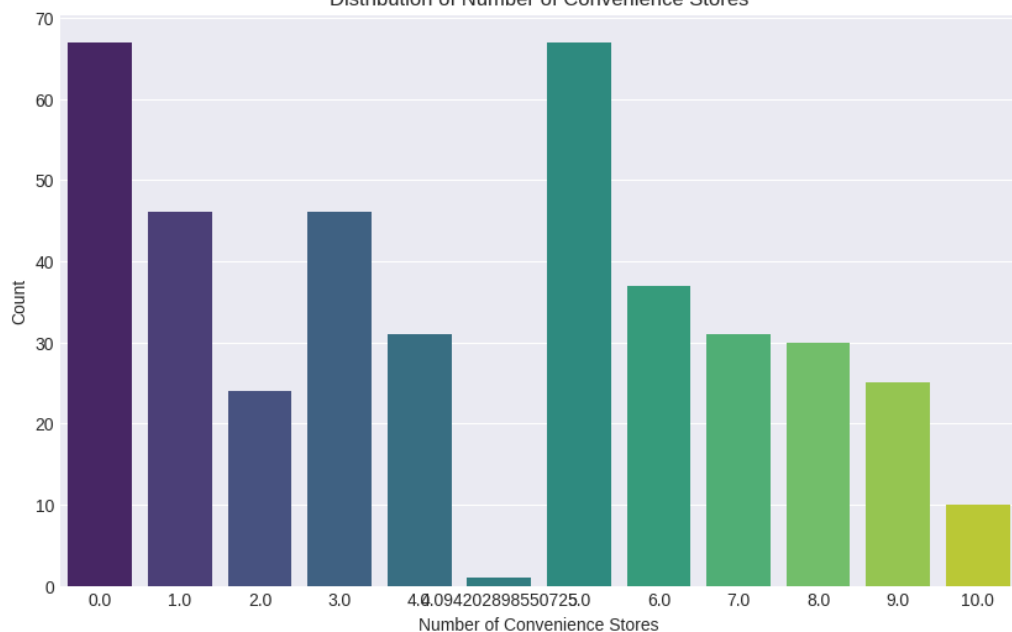




```
/tmp/ipykernel_1786/2059081203.py:66: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and w
```

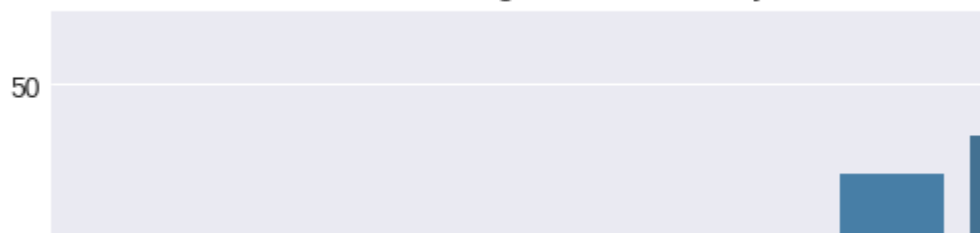
```
sns.countplot(x='X4 number of convenience stores', data=df,
Distribution of Number of Convenience Stores
```



```
/tmp/ipykernel_1786/2059081203.py:75: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and w
```

```
sns.barplot(x='X4 number of convenience stores', y='Y house
Average House Price by Number of Cc
```



Based on the exploratory analysis and visualizations:

(a) The attribute that has the strongest positive correlation with

house price: Looking at the correlation heatmap, 'X4 number of convenience stores' has the strongest positive correlation with 'Y house price of unit area' at approximately **0.57**. This suggests that houses closer to more convenience stores tend to have higher unit prices.

(b) The attribute that has the strongest negative correlation with

house price: From the correlation heatmap, 'X3 distance to the nearest MRT station' has the strongest negative correlation with 'Y house price of unit area' at approximately **-0.67**. This indicates that as the distance to the nearest MRT station increases, the house price tends to decrease.

(c) Any possible outliers in the dataset:

- **House Price:** The box-and-whisker plot for 'Y house price of unit area' clearly shows several data points above the upper whisker, indicating potential outliers with unusually high house prices, even after capping the maximum at 110.0. The maximum value in the box plot is around 110, which corresponds to the capping performed earlier, but values around the 80-110 range appear as distinct points above the main distribution. The scatter plots also show some points with high house prices that are somewhat separated from the main clusters.
- **Distance to MRT Station:** The scatter plot of 'House Price vs.

Distance to Nearest MRT Station' shows some houses with very high distances to the MRT station (e.g., above 4000 meters) that still have moderate house prices, which might be considered outliers depending on further domain knowledge.

These outliers should be further investigated to determine if they are data entry errors, rare events, or genuinely exceptional cases that could provide valuable insights.

[Colab paid products](#) - [Cancel contracts here](#)